## CSA0412-OPERATING SYSTEMS

**EXP 12-Implement read and write using dining philosopher**

```
#include <stdio.h> #include <pthread.h> #include <semaphore.h> #include <unistd.h>

#define N 5

sem_t fork_sem[N];

void *philosopher(void *arg) { int id = *(int *)arg; int left = id; int right = (id + 1) % N;

printf("Philosopher %d is thinking\n", id + 1);
sleep(1);

sem_wait(&fork_sem[left]);
sem_wait(&fork_sem[right]);

printf("Philosopher %d is eating\n", id + 1);
sleep(1);

sem_post(&fork_sem[right]);
sem_post(&fork_sem[left]);

printf("Philosopher %d has finished eating\n", id + 1);

return NULL;


}

int main() { pthread_t p[N]; int id[N];

for (int i = 0; i < N; i++)
   sem_init(&fork_sem[i], 0, 1);

for (int i = 0; i < N; i++) {
   id[i] = i;
   pthread_create(&p[i], NULL, philosopher, &id[i]);
}

for (int i = 0; i < N; i++)
   pthread_join(p[i], NULL);
```

```
for (int i = 0; i < N; i++)
    sem_destroy(&fork_sem[i]);

return 0;


}
```

